

Double-click (or enter) to edit

```
!pip install transformers datasets torch scikit-learn matplotlib seaborn pandas numpy tqdm --quiet
```

```
from google.colab import files
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import torch
import warnings
import os
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.preprocessing import LabelEncoder
from torch.utils.data import Dataset, DataLoader
from transformers import (
    BertTokenizer, BertForSequenceClassification,
    get_linear_schedule_with_warmup
)
from torch.optim import AdamW
from tqdm import tqdm

warnings.filterwarnings('ignore')
sns.set_theme(style='darkgrid', palette='muted')
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
```

Device: cuda

```
df = pd.read_csv('twitter_training.csv', header=None, names=['id', 'topic', 'sentiment', 'text'])
df.dropna(subset=['text'], inplace=True)
df['text'] = df['text'].astype(str).str.strip()
df = df[df['text'].str.len() > 2].reset_index(drop=True)
print(df.shape)
df.head()
```

(73140, 4)

	id	topic	sentiment	text
0	2401	Borderlands	Positive	im getting on borderlands and i will murder yo...
1	2401	Borderlands	Positive	I am coming to the borders and I will kill you...
2	2401	Borderlands	Positive	im getting on borderlands and i will kill you ...
3	2401	Borderlands	Positive	im coming on borderlands and i will murder you...
4	2401	Borderlands	Positive	im getting on borderlands 2 and i will murder ...

```
PALETTE = {
    'Positive': '#2ecc71',
    'Negative': '#e74c3c',
    'Neutral': '#3498db',
    'Irrelevant': '#95a5a6'
}

counts = df['sentiment'].value_counts()
colors = [PALETTE[s] for s in counts.index]

fig, axes = plt.subplots(1, 3, figsize=(18, 5), facecolor='#1a1a2e')
for ax in axes:
    ax.set_facecolor('#16213e')
    ax.tick_params(colors='white')
    for spine in ax.spines.values():
        spine.set_edgecolor('#0f3460')

axes[0].bar(counts.index, counts.values, color=colors, edgecolor='white', linewidth=0.5)
```

```

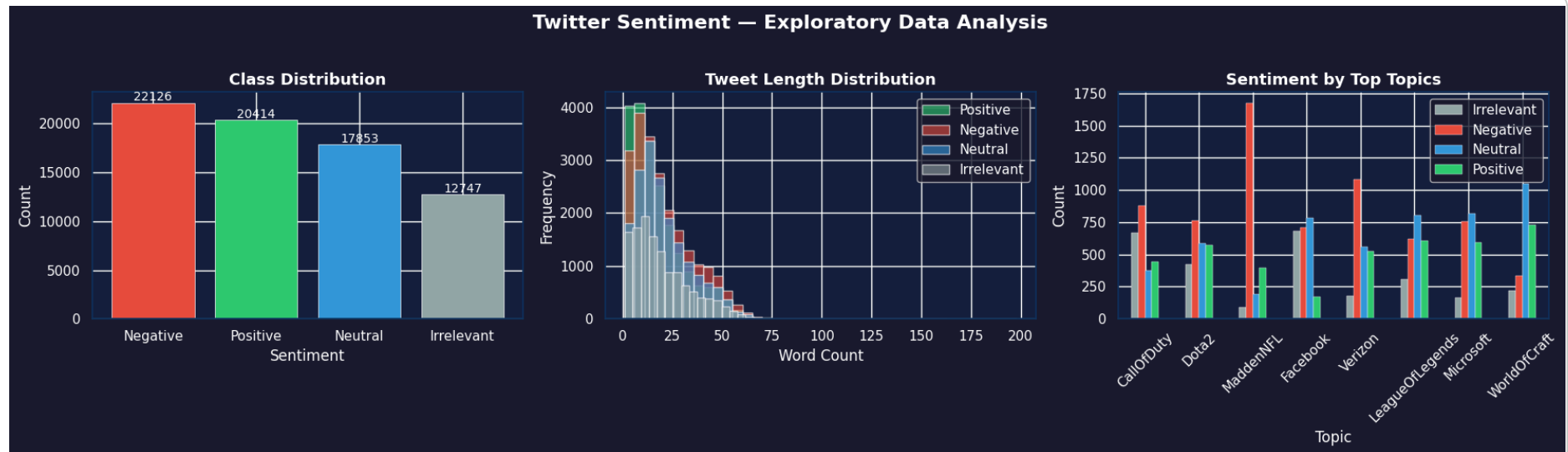
axes[0].set_title('Class Distribution', color='white', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Sentiment', color='white')
axes[0].set_ylabel('Count', color='white')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 200, str(v), ha='center', color='white', fontsize=10)

df['text_len'] = df['text'].str.split().str.len()
for sent, color in PALETTE.items():
    subset = df[df['sentiment'] == sent]['text_len']
    axes[1].hist(subset, bins=40, alpha=0.6, label=sent, color=color)
axes[1].set_title('Tweet Length Distribution', color='white', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Word Count', color='white')
axes[1].set_ylabel('Frequency', color='white')
axes[1].legend(facecolor='#1a1a2e', labelcolor='white')

topic_sent = df.groupby(['topic', 'sentiment']).size().unstack(fill_value=0)
top_topics = df['topic'].value_counts().head(8).index
topic_sent.loc[top_topics].plot(
    kind='bar', ax=axes[2],
    color=[PALETTE[c] for c in topic_sent.columns],
    edgecolor='white', linewidth=0.3
)
axes[2].set_title('Sentiment by Top Topics', color='white', fontsize=13, fontweight='bold')
axes[2].set_xlabel('Topic', color='white')
axes[2].set_ylabel('Count', color='white')
axes[2].tick_params(axis='x', rotation=45)
axes[2].legend(facecolor='#1a1a2e', labelcolor='white')

fig.suptitle('Twitter Sentiment – Exploratory Data Analysis', color='white',
            fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('eda_analysis.png', dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
plt.show()
print('Saved: eda_analysis.png')

```



Saved: eda_analysis.png

```
le = LabelEncoder()
df['label'] = le.fit_transform(df['sentiment'])
NUM_CLASSES = len(le.classes_)
print('Classes:', dict(zip(le.classes_, le.transform(le.classes_))))

train_texts, val_texts, train_labels, val_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(),
    test_size=0.15, random_state=42, stratify=df['label']
)
print(f'Train: {len(train_texts)} | Val: {len(val_texts)}')
```

```
Classes: {'Irrelevant': np.int64(0), 'Negative': np.int64(1), 'Neutral': np.int64(2), 'Positive': np.int64(3)}
Train: 62169 | Val: 10971
```

```
MODEL_NAME = 'bert-base-uncased'
MAX_LEN     = 128
BATCH_SIZE  = 32
EPOCHS      = 3
LR          = 2e-5

tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)
```

```

class TwitterDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts      = texts
        self.labels      = labels
        self.tokenizer   = tokenizer
        self.max_len     = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        enc = self.tokenizer(
            self.texts[idx],
            max_length=self.max_len,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
        return {
            'input_ids':      enc['input_ids'].squeeze(),
            'attention_mask': enc['attention_mask'].squeeze(),
            'label':          torch.tensor(self.labels[idx], dtype=torch.long)
        }

train_ds = TwitterDataset(train_texts, train_labels, tokenizer, MAX_LEN)
val_ds   = TwitterDataset(val_texts,   val_labels,   tokenizer, MAX_LEN)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
print(f'Train batches: {len(train_loader)} | Val batches: {len(val_loader)}')

```

tokenizer_config.json: 100%

48.0/48.0 [00:00<00:00, 1.44kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable

vocab.txt: 232k/? [00:00<00:00, 3.06MB/s]

tokenizer.json: 466k/? [00:00<00:00, 3.48MB/s]

Train batches: 1943 | Val batches: 343

```
model = BertForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=NUM_CLASSES
).to(DEVICE)

optimizer = AdamW(model.parameters(), lr=LR, weight_decay=0.01)
total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_steps),
    num_training_steps=total_steps
)
print(f'Total steps: {total_steps}')
```

config.json: 100%

570/570 [00:00<00:00, 12.8kB/s]

model.safetensors: 100%

440M/440M [00:03<00:00, 221MB/s]

Loading weights: 100%

199/199 [00:00<00:00, 413.15it/s, Materializing param=bert.pooler.dense.weight]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Total steps: 5829

```
def train_epoch(model, loader, optimizer, scheduler, device):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for batch in tqdm(loader, desc='Training'):
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)
        labels = batch['label'].to(device)
        optimizer.zero_grad()
        out = model(input_ids=ids, attention_mask=mask, labels=labels)
        out.loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        total_loss += out.loss.item()
        preds = out.logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return total_loss / len(loader), correct / total
```

```

def eval_epoch(model, loader, device):
    model.eval()
    total_loss, correct, total = 0, 0, 0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for batch in tqdm(loader, desc='Evaluating'):
            ids = batch['input_ids'].to(device)
            mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)
            out = model(input_ids=ids, attention_mask=mask, labels=labels)
            total_loss += out.loss.item()
            preds = out.logits.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    return total_loss / len(loader), correct / total, all_preds, all_labels

```

```

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
best_val_acc = 0

for epoch in range(1, EPOCHS + 1):
    print(f'\nEpoch {epoch}/{EPOCHS}')
    tr_loss, tr_acc = train_epoch(model, train_loader, optimizer, scheduler, DEVICE)
    vl_loss, vl_acc, preds, labels_true = eval_epoch(model, val_loader, DEVICE)
    history['train_loss'].append(tr_loss)
    history['val_loss'].append(vl_loss)
    history['train_acc'].append(tr_acc)
    history['val_acc'].append(vl_acc)
    print(f' Train Loss: {tr_loss:.4f} | Train Acc: {tr_acc:.4f}')
    print(f' Val Loss: {vl_loss:.4f} | Val Acc: {vl_acc:.4f}')
    if vl_acc > best_val_acc:
        best_val_acc = vl_acc
        torch.save(model.state_dict(), 'best_bert_model.pt')
        print(' Best model saved.')

final_preds, final_labels = preds, labels_true

```



```
Epoch 1/3
Training: 100%|██████████| 1943/1943 [20:21<00:00, 1.59it/s]
Evaluating: 100%|██████████| 343/343 [01:10<00:00, 4.88it/s]
  Train Loss: 0.8990 | Train Acc: 0.6333
  Val Loss: 0.5890 | Val Acc: 0.7794
  Best model saved.
```

```
Epoch 2/3
Training: 100%|██████████| 1943/1943 [20:25<00:00, 1.59it/s]
Evaluating: 100%|██████████| 343/343 [01:10<00:00, 4.87it/s]
  Train Loss: 0.3961 | Train Acc: 0.8601
  Val Loss: 0.3250 | Val Acc: 0.8907
  Best model saved.
```

```
Epoch 3/3
Training: 100%|██████████| 1943/1943 [20:25<00:00, 1.59it/s]
Evaluating: 100%|██████████| 343/343 [01:10<00:00, 4.88it/s]
  Train Loss: 0.1827 | Train Acc: 0.9372
  Val Loss: 0.2843 | Val Acc: 0.9130
  Best model saved.
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5), facecolor='#1a1a2e')
epochs_range = range(1, EPOCHS + 1)

for ax in axes:
    ax.set_facecolor('#16213e')
    ax.tick_params(colors='white')
    for spine in ax.spines.values():
        spine.set_edgecolor('#0f3460')

axes[0].plot(epochs_range, history['train_loss'], 'o-', color='#e74c3c', label='Train Loss', linewidth=2)
axes[0].plot(epochs_range, history['val_loss'], 's-', color='#3498db', label='Val Loss', linewidth=2)
axes[0].set_title('Loss per Epoch', color='white', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Epoch', color='white')
axes[0].set_ylabel('Loss', color='white')
axes[0].legend(facecolor='#1a1a2e', labelcolor='white')

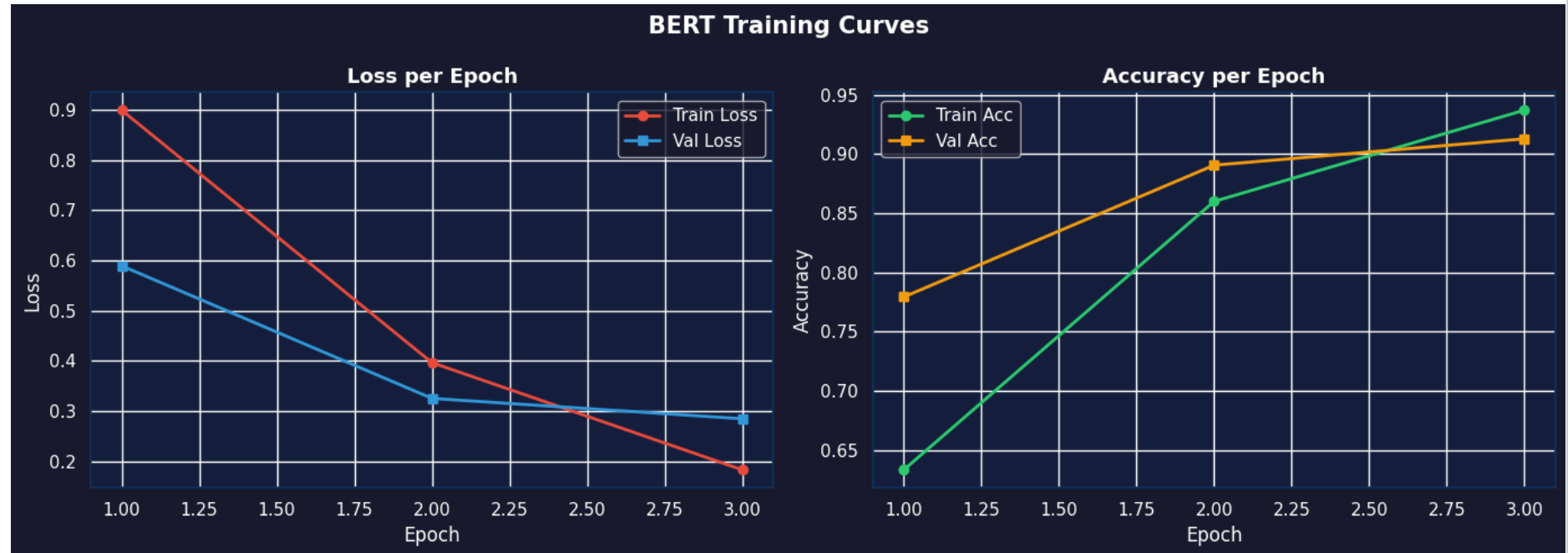
axes[1].plot(epochs_range, history['train_acc'], 'o-', color='#2ecc71', label='Train Acc', linewidth=2)
axes[1].plot(epochs_range, history['val_acc'], 's-', color='#f39c12', label='Val Acc', linewidth=2)
axes[1].set_title('Accuracy per Epoch', color='white', fontsize=13, fontweight='bold')
```

```

axes[1].set_xlabel('Epoch', color='white')
axes[1].set_ylabel('Accuracy', color='white')
axes[1].legend(facecolor='#1a1a2e', labelcolor='white')

fig.suptitle('BERT Training Curves', color='white', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
plt.show()
print('Saved: training_curves.png')

```



Saved: training_curves.png

```

model.load_state_dict(torch.load('best_bert_model.pt', map_location=DEVICE))
_, val_acc_best, final_preds, final_labels = eval_epoch(model, val_loader, DEVICE)

print('Classification Report')
print(classification_report(final_labels, final_preds, target_names=le.classes_))

cm = confusion_matrix(final_labels, final_preds)

```

```
fig, ax = plt.subplots(figsize=(8, 6), facecolor='#1a1a2e')
ax.set_facecolor('#16213e')
sns.heatmap(
    cm, annot=True, fmt='d', cmap='YlOrRd',
    xticklabels=le.classes_, yticklabels=le.classes_,
    linewidths=0.5, linecolor='#0f3460', ax=ax
)
ax.set_title('Confusion Matrix – Best BERT Model', color='white', fontsize=13, fontweight='bold')
ax.set_xlabel('Predicted', color='white')
ax.set_ylabel('Actual', color='white')
ax.tick_params(colors='white')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight', facecolor='#1a1a2e')
plt.show()
print('Saved: confusion_matrix.png')
```


Evaluating: 100%|██████████| 343/343 [01:10<00:00, 4.86it/s]

Classification Report

precision	recall	f1-score	support
-----------	--------	----------	---------

```
from sklearn.metrics import precision_recall_fscore_support

precision, recall, f1, support = precision_recall_fscore_support(
    final_labels, final_preds, labels=range(NUM_CLASSES)
)

metrics_df = pd.DataFrame({
    'Class': le.classes_,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
})

x = np.arange(NUM_CLASSES)
width = 0.25

fig, ax = plt.subplots(figsize=(10, 6), facecolor='#1a1a2e')
ax.set_facecolor('#16213e')
ax.tick_params(colors='white')
for spine in ax.spines.values():
    spine.set_edgecolor('#0f3460')

ax.bar(x - width, precision, width, label='Precision', color='#3498db', edgecolor='white', linewidth=0.4)
ax.bar(x, recall, width, label='Recall', color='#2ecc71', edgecolor='white', linewidth=0.4)
ax.bar(x + width, f1, width, label='F1-Score', color='#e74c3c', edgecolor='white', linewidth=0.4)

ax.set_xticks(x)
ax.set_xticklabels(le.classes_, color='white')
ax.set_ylim(0, 1.05)
ax.set_title('Per-Class Metrics - BERT', color='white', fontsize=13, fontweight='bold')
ax.set_ylabel('Score', color='white')
ax.legend(facecolor='#1a1a2e', labelcolor='white')

for bars in [ax.patches[i::3] for i in range(3)]:
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.2f}', ha='center', va='bottom',
```

```
color='white', fontsize=8)
```

```
plt.tight_lavout()
```